

JavaScript DOM Manipulation — Study Notes

Expanded edition · based on practice code: `app.js` / `index.html` / `styles.css`

1. Selecting Elements

Before JS can change anything on a page, it has to grab a reference to the HTML element first. That reference is just a JS object representing that piece of the page — once you have it stored in a variable, you can read or change its properties.

`getElementById()`

Returns exactly one element — the one whose id attribute matches. Since ids are supposed to be unique on a page, this always returns a single object (or null if nothing matches).

```
var a = document.getElementById("text");
```

`getElementsByName()`

Returns an `HTMLCollection` — an array-like list of every element with that tag, in document order. It is NOT a real array (no `.map/.filter` directly), and it is "live": if elements matching the tag are added or removed from the page later, the collection updates automatically without you having to re-query it.

```
var p = document.getElementsByTagName("p"); // p[0], p[1], p[2]... and p.length
```

Related methods worth knowing: `getElementsByClassName("name")` works the same way but matches by class, and `querySelector("p.text_1")` / `querySelectorAll(".text_1")` let you use full CSS selectors — `querySelector` returns the first match, `querySelectorAll` returns all matches as a (static, non-live) `NodeList`.

Why defer matters here

In `app.js`, this line sits outside any function, at the top level of the file:

```
var a = document.getElementById("para"); console.log(a.innerHTML);
```

Top-level code runs immediately, in order, the moment the browser executes the script — it doesn't wait for a button click or any event. Normally that's a problem: if the browser hasn't finished building the page yet, `#para` wouldn't exist and `getElementById` would return null, causing a crash when `.innerHTML` is accessed next.

```
<script defer src="./app.js"></script>
```

The `defer` attribute on the `<script>` tag tells the browser: download this file in the background, but don't run it until the entire HTML document has been parsed. That guarantees every element — including `#para` — already exists in the DOM by the time `app.js` executes, which is exactly why this code works. Without `defer` (or with the script tag placed in the `<head>` without `defer/async`), this line would throw "Cannot read properties of null".

2. Event Handling

Events are the browser telling your code "something happened" — a click, a key press, a field losing focus, a mouse entering an element. Attaching a handler means: run this function when that event happens.

Inline HTML attributes (the style used in this project)

Attribute	Fires when...	Typical use
onclick	element is clicked (mouse or tap)	buttons, links, images
onfocus	an input/field becomes active (clicked into or tabbed to)	highlight the active field
onblur	an input/field loses focus (user clicks elsewhere)	remove highlight, validate, read final value
onmouseover	cursor enters the element's area	hover effects, tooltips
onmouseout / onmouseleave	cursor leaves the element's area	reverse a hover effect
onchange	a form field's value is committed (e.g. after blur, or selection of a different option)	react to finalized value

Passing arguments to a handler

```
<button onclick="test2(`Daniyal`)">Click me</button> <button onclick="test2(`Ahsan`)">Click me</button>
function test2(a) { console.log("Hello " + a); }
```

Both buttons call the same function, but each passes a different literal string. The backticks here are just a JavaScript string (a template literal) — they could equally be single or double quotes. Because the HTML attribute itself is written inside double quotes, backticks (or single quotes) avoid a quoting conflict. This is a simple but important pattern: one reusable function, many callers, each supplying its own data through parameters.

focus/blur pair as a mini state machine

```
<input id="text" onfocus="changebg1()" onblur="changebg()">
function changebg1() { // on focus
  var a = document.getElementById("text");
  a.style.backgroundColor = "yellow";
}
function changebg() { // on blur
  var a = document.getElementById("text");
  a.style.backgroundColor = "white";
}
```

This is essentially a two-state toggle driven by two different events instead of one click handler: "entered" state (yellow) and "left" state (white). It's a common UI pattern — visually indicating which field is currently active.

Inline handlers vs addEventListener

The project uses inline attributes (onclick="...") directly in the HTML. This works and is easy to read while learning, but has real downsides once projects grow:

- Mixes structure (HTML) with behavior (JS) in the same file, making both harder to maintain.
- Only one inline handler per event per element — you can't easily attach two separate onclick behaviors to the same button.
- The function has to be a global function (attached to window) for the inline attribute to find it.

The more scalable approach, addEventListener, looks like this:

```
// in app.js, no HTML attribute needed
var btn = document.getElementById("myButton");
btn.addEventListener("click", function () { console.log("clicked!"); });
```

This keeps all behavior in the JS file, and lets you attach multiple listeners to the same element/event without overwriting each other.

3. Changing Inline Styles

The **.style** property gives you direct read/write access to an element's inline style attribute (the `style="..."` you'd otherwise write by hand in HTML). Every CSS property is available as a JS property on `.style`, but the naming changes slightly.

CSS-to-JS naming: camelCase

CSS property	JS property (<code>.style.____</code>)
background-color	backgroundColor
font-size	fontSize
border-radius	borderRadius
text-align	textAlign

Any CSS property with a hyphen drops the hyphen and capitalizes the next word — this mirrors how JS variable names can't contain hyphens (hyphens would be parsed as subtraction).

.style vs **.className** — an important trade-off

`.style` changes ONE property at a time directly, bypassing your CSS file entirely — quick for a one-off change, but styling logic ends up scattered through JS instead of living in `styles.css`. `.className` (covered in section 6) instead swaps which CSS class is applied, keeping the actual colors/sizes defined in one place. For anything beyond a quick toggle, `className/classList` is usually the better habit.

4. Reading and Writing Input Values

Form fields (`<input>`, `<textarea>`, `<select>`) don't store their current content in `.innerHTML` — they use a separate **.value** property instead.

```
<input type="text" id="test3"> function getvalue() { var a =
document.getElementById("test3"); console.log(a.value); // reads whatever the user typed }
function setvalue() { var a = document.getElementById("test3"); a.value = "Hello world"; //
overwrites the field }
```

Why a separate property? An `<input>` is a self-closing tag with no inner HTML content to read — its "value" lives in the `value` attribute/property instead. This is a very common early mix-up: trying `a.innerHTML` on an input does not work the way you'd expect (inputs don't have child content the way a `<p>` or `<div>` does).

`newf()` from the code follows the same pattern on `blur` — reading the value only once the user has finished typing and clicked away:

```
function newf() { var a = document.getElementById("text1"); console.log(a.value); }
```

5. innerHTML — Reading and Replacing Content

.innerHTML gets or sets everything between an element's opening and closing tag, as an HTML string. Unlike `.value`, it works on any element (not just form fields), and — crucially — it understands HTML markup, not just plain text.

```
function chnageText() { var a = document.getElementById("para"); a.innerHTML = "<b> Hello  
</b>"; }
```

Because the string contains `<b>` tags, the browser parses it as HTML and renders the word bold — it does not display the literal characters `"<b>".` Setting the same text through a property that treats it as plain text (`.textContent = "<b> Hello </b>"`) would show the tags literally on the page instead of rendering them.

innerHTML vs textContent

	innerHTML	textContent
Parses HTML tags?	Yes	No — shown as literal text
Faster?	Slower (has to parse markup)	Faster
Safe with user input?	Risky — can execute scripts (XSS)	Safe — always treated as text

Security note: the examples in this project (fixed strings the developer wrote) are perfectly safe. The risk only appears if you ever do something like `element.innerHTML = someUserInput` — if `someUserInput` contains a `<script>` tag or an event attribute, the browser will execute it. That's called Cross-Site Scripting (XSS). Rule of thumb: use `innerHTML` for markup you wrote yourself; use `textContent` for anything coming from a user.

6. className — Swapping and Adding CSS Classes

.className reads or writes an element's entire class attribute as a single space-separated string. It's the JS equivalent of directly editing `class="..."` in the HTML.

```
function chnagecolor() { var a = document.getElementById("para"); a.className = "red"; // REPLACES all existing classes } function color() { var i = document.getElementById("fff"); i.className += " hhh"; // APPENDS "hhh" to existing classes }
```

= (overwrite) vs += (append) — walk through it

Say #fff starts with `class="text_1"`. Because `color()` uses `+=`, `className` becomes `"text_1 hhh"` — both the border style from `.text_1` and the blue background from `.hhh` apply together. If `chnagecolor()` used `className += "red"` instead of `=`, #para would end up with whatever classes it already had, plus "red" tacked on.

The leading space in `" hhh"` is not optional. `className += "hhh"` (no space) on an element that already has `class="text_1"` would silently produce `class="text_1hhh"` — a single class name that matches neither `.text_1` nor `.hhh` in the stylesheet. This is a very easy bug to introduce and a very confusing one to debug, since nothing throws an error — the style just quietly fails to apply.

The modern alternative: `classList`

```
i.classList.add("hhh"); // add one class, no space bugs possible i.classList.remove("hhh"); // remove one class i.classList.toggle("hhh"); // add if missing, remove if present i.classList.contains("hhh"); // true/false check
```

`classList` exists specifically to avoid the `className` string-concatenation problems above. `add/remove/toggle` only ever affect the one class you name, leaving every other class on the element untouched — no manual spacing, no risk of merging class names together.

7. Toggling an Attribute (Image Swap)

```
function ccc() { var c = document.getElementById("ccc"); var image1 = "https://.../car1.jpg"; var image2 = "https://.../car2.jpg"; if (c.src === image1) { c.src = image2; } else { c.src = image1; } }
```

This checks the image's CURRENT source and flips to the other one. The `else` branch also covers the very first click, when `.src` might be neither `image1` nor `image2` (e.g. a base64-encoded placeholder, as in this project) — in that case it simply falls into showing `image1`.

The same `if/else` toggle shape generalizes well beyond images — showing/hiding a menu, switching a button's label between "Play"/"Pause", or flipping a dark-mode class on/off all follow this exact check-current-state-then-flip pattern.

A subtlety worth knowing: reading `.src` back from the DOM after setting it often returns the browser's fully resolved absolute URL, not necessarily the exact string you assigned — this can occasionally make `===` comparisons fail unexpectedly if the assigned string wasn't already a full URL. In this project both `image1` and `image2` are already full `https://` URLs, so it isn't an issue here, but it's a common gotcha to watch for.

8. Looping Over Multiple Elements

```
function chnagebg() { var p = document.getElementsByTagName("p"); for (var i = 0; i < p.length; i++) { p[i].className = "green"; } }
```

`getElementsByTagName` returns a collection, not one element — `p.className` would not work, because a collection itself has no `className` property. Each individual element inside the collection is accessed by index (`p[0]`, `p[1]`, ...), which is why the loop is necessary: it visits every `<p>` one at a time and applies the class to each.

`p.length` works the same way an array's `.length` would — it's the count of items in the collection, used here as the loop's stopping condition.

A modern alternative, once comfortable with array methods: `Array.from(p).forEach(el => el.className = "green");` — this converts the `HTMLCollection` into a real array first, unlocking `forEach/map/filter`.

Quick Reference Summary

Property/Method	Purpose
<code>getElementById(id)</code>	select one element by id
<code>getElementsByTagName(tag)</code>	select all elements of a tag (live collection)
<code>querySelector / querySelectorAll</code>	select using any CSS selector
<code>.style.property</code>	change one inline CSS property (camelCase name)
<code>.value</code>	read/write a form field's content
<code>.innerHTML</code>	read/write an element's HTML content (parses tags)
<code>.textContent</code>	read/write plain text only (no tag parsing)
<code>.className</code>	read/write an element's class attribute (overwrites)
<code>.className +=</code>	append a class as a string (watch the leading space)
<code>.classList.add/remove/toggle</code>	safely add/remove one class
<code>.src</code>	read/write an image or media source
<code>addEventListener(event, fn)</code>	attach a handler from JS instead of inline HTML

Things to Clean Up / Practice Next

- Replace raw `className` string concatenation with `classList.add()` / `.remove()` / `.toggle()`.
- Move inline `onclick/onfocus/onblur` attributes into `addEventListener()` calls in `app.js` — better separation of HTML structure and JS behavior, and allows multiple listeners per element.
- Fix inconsistent function-name spelling (`chnageText`, `chnagecolor`, `chnagebg`) — small typos add up as a project grows and make functions harder to find/reuse.
- Try replacing `var` with `let/const` — `const` for values that never get reassigned (like element references), `let` for ones that do. This avoids some of `var`'s quirks around scope (`var` is function-scoped, not block-scoped).
- Practice rewriting the `getElementsByTagName` loop using `Array.from(p).forEach(...)` once comfortable with array methods.
- Try converting one inline-`onclick` example to `addEventListener` as a small refactor exercise, to feel the difference directly.